

INFO1195 | Actividad 4

Procesamiento de imágenes con CUDA C++ clásico, CUDA Tile C++, CUDA Python y profiling con herramientas CUDA

Fecha de entrega	Domingo 05 de julio de 2026
Porcentaje de la actividad	30 % de la asignatura

1. Objetivo de la actividad

Implementar, analizar y comparar una pipeline de procesamiento de imágenes usando CUDA C++ clásico, CUDA Tile C++, CUDA Python y herramientas de medición de CUDA. La actividad busca evidenciar comprensión del paralelismo en GPU, el uso de hilos, bloques y tiles, la organización de memoria, las transferencias host-device, la validación, el profiling y la comparación entre referencia CPU, CUDA clásico, CUDA Tile y cuTile Python.

Para la nota se considera lo siguiente:

- **Informe técnico**, 70 %: problema, metodología, resultados, rendimiento, memoria, profiling y conclusiones.
- **Código**, 30 %: **implementación funcional, estructura del proyecto, reproducibilidad, correctitud, scripts de ejecución, resultados generados, calidad técnica del código y consistencia con el informe. Nota: si el código no compila o no ejecuta la pipeline mínima, tanto el código como el informe recibirán nota mínima.**

Nota: Se descontarán puntos si se encuentra información en el informe que no se encuentre en el código, o viceversa.

2. Contexto del problema

El procesamiento de imágenes permite aplicar una misma operación sobre muchos píxeles, por lo que es adecuado para estudiar paralelismo en GPU. Filtros como desenfoque gaussiano, Sobel y redimensionamiento bilineal permiten analizar rendimiento, memoria y regularidad del cálculo.

CUDA C++ clásico exige controlar explícitamente hilos, bloques, grilla, warps e indexación de memoria. En cambio, CUDA Tile C++ permite expresar la solución mediante tiles o bloques lógicos de datos, facilitando el trabajo sobre localidad, bordes y organización de la memoria.

La actividad actualiza este dominio incorporando CUDA C++ clásico, CUDA Tile C++, cuTile Python, CUDA Python y herramientas de medición/profiling de CUDA. El grupo deberá demostrar comprensión del algoritmo, de las diferencias entre modelos de programación GPU y del impacto real de sus decisiones de implementación en el rendimiento.

3. Fundamentos matemáticos mínimos

Antes de programar la pipeline, el grupo debe comprender que cada etapa calcula nuevos valores de salida a partir de píxeles vecinos o de coordenadas proyectadas. Como la misma regla se repite para muchos píxeles, el problema es adecuado para paralelizarlo por tiles en GPU. **El estudiante deberá investigar y complementar estos 3 fundamentos matemáticos, para poder implementar correctamente estos filtros.**

3.1 Filtro de desenfoque gaussiano

El filtro gaussiano suaviza una imagen reduciendo ruido y variaciones bruscas de intensidad. Para cada píxel se calcula un promedio ponderado de sus vecinos: los píxeles cercanos al

centro tienen mayor peso y los más lejanos tienen menor influencia. Esta ponderación se obtiene desde una distribución gaussiana.

$$G(x, y) = (1 / (2\pi\sigma^2)) * \exp(-(x^2 + y^2) / (2\sigma^2))$$

En la práctica, la fórmula se convierte en una máscara o kernel, por ejemplo 5x5 o 9x9. Los valores del kernel deben normalizarse para que la suma sea 1 y así evitar que la imagen quede artificialmente más clara u oscura.

La implementación puede realizarse como convolución 2D directa o como filtro separable. En la versión directa, cada píxel requiere revisar $k \times k$ vecinos. En la versión separable, se aplica primero un filtro horizontal y luego uno vertical, reduciendo el costo aproximado a $2 \times k$ operaciones por píxel. Esta diferencia permite discutir uso de memoria, reutilización de datos y tamaño de tile.

3.2 Operador de Sobel

El operador de Sobel detecta bordes midiendo cambios de intensidad entre píxeles vecinos. Normalmente se aplica sobre una imagen en escala de grises o luminancia. Para cada píxel se revisa una vecindad 3x3 y se calculan dos gradientes: uno en dirección X y otro en dirección Y.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

G_x responde principalmente a cambios verticales de intensidad, mientras que G_y responde a cambios horizontales. Luego se calcula la magnitud del borde y se ajusta el resultado al rango de la imagen, por ejemplo 0 a 255.

$$G = \sqrt{G_x^2 + G_y^2}$$

También se puede usar una aproximación como $|G_x| + |G_y|$ si se justifica. El grupo debe explicar cómo convierte RGB a luminancia, cómo accede a los vecinos, cómo trata los bordes y cómo valida que los bordes obtenidos sean coherentes con la referencia CPU.

3.3 Interpolación bilineal

La interpolación bilineal permite redimensionar una imagen calculando cada píxel de salida a partir de una coordenada fraccional en la imagen original. A diferencia de Gaussian y Sobel, esta etapa no solo revisa vecinos inmediatos, sino que proyecta la posición del píxel de salida hacia la imagen de entrada.

Para una coordenada fraccional (x, y) , se identifican los cuatro píxeles más cercanos: A, B, C y D. Luego se calculan las distancias relativas dx y dy , y se combinan los cuatro valores según su cercanía al punto proyectado.

$$\text{valor} = A \cdot (1 - dx) \cdot (1 - dy) + B \cdot dx \cdot (1 - dy) + C \cdot (1 - dx) \cdot dy + D \cdot dx \cdot dy$$

Esta etapa exige definir claramente la relación entre dimensiones de entrada y salida, el cálculo de coordenadas, el redondeo o truncamiento usado, y el tratamiento de bordes cuando la coordenada cae cerca del límite de la imagen.

4. Desarrollo

Cada grupo deberá implementar una pipeline de procesamiento de imágenes RGB compuesta por tres etapas principales: filtro gaussiano, operador de Sobel y redimensionamiento bilineal. La solución debe incluir referencia CPU, implementación CUDA C++ clásica sin Tile, implementación CUDA Tile C++ y una etapa en cuTile Python.

La actividad no consiste solo en generar una imagen procesada. El grupo deberá demostrar que comprende cómo se organiza la imagen en memoria, cómo se divide el trabajo en hilos, bloques y tiles, cómo se manejan los bordes, qué datos se transfieren entre CPU y GPU, cómo se valida la correctitud y cómo se mide el rendimiento con herramientas de CUDA.

4.1 Etapa 1: filtro gaussiano

El grupo deberá implementar el desenfoque gaussiano sobre la imagen de entrada. Esta etapa debe permitir observar cómo una convolución regular puede aprovechar la GPU, especialmente cuando se procesan imágenes medianas o grandes.

- Implementar versión secuencial en CPU.
- Implementar versión CUDA C++ clásica sin CUDA Tile, usando kernels con hilos, bloques, grilla e indexación explícita.
- Implementar versión en CUDA Tile C++.
- Usar al menos dos tamaños de kernel o dos valores de sigma.
- Definir y aplicar una estrategia de bordes: clamp, réplica, cero o exclusión controlada.
- Comparar las salidas CUDA C++ clásica y CUDA Tile C++ contra la CPU usando una métrica numérica y una validación visual.
- Medir el tiempo de ejecución con CUDA Events, Nsight Compute, Nsight Systems o herramientas equivalentes de CUDA, e indicar si se usó convolución 2D directa o filtro separable.

Se recomienda la versión separable porque permite discutir reducción de operaciones, reutilización de datos y diferencias entre una pasada horizontal y una pasada vertical.

4.2 Etapa 2: operador de Sobel

El grupo deberá implementar la detección de bordes mediante Sobel. Esta etapa debe mostrar cómo se accede a una vecindad 3x3 por cada píxel y cómo se calculan gradientes a partir de una imagen en escala de grises o luminancia.

- Convertir RGB a luminancia o justificar el canal utilizado.
- Calcular Gx y Gy usando las máscaras de Sobel.
- Calcular la magnitud del borde y normalizar o saturar el resultado al rango permitido.
- Implementar versión CPU, versión CUDA C++ clásica y versión CUDA Tile C++.
- Explicar el acceso a píxeles vecinos, tratamiento de bordes e indexación.
- Validar visual y numéricamente contra la referencia CPU.

4.3 Etapa 3: redimensionamiento bilineal

El grupo deberá implementar resize mediante interpolación bilineal. Esta etapa debe mostrar cómo cada píxel de salida se calcula desde una posición fraccional de la imagen original y cómo ese cálculo se paraleliza en GPU.

- Implementar versión CPU, versión CUDA C++ clásica y versión CUDA Tile C++.
- Usar al menos dos factores de escala, por ejemplo 0.5x y 1.75x. El código debe permitir modificar el factor de escala mediante argumento, archivo de configuración o variable claramente documentada.
- Calcular coordenadas fraccionales para cada píxel de salida.
- Interpoliar usando los cuatro píxeles vecinos de la imagen original.
- Manejar bordes y dimensiones no divisibles por el tamaño del tile.
- Comparar resultados visuales y métricas de error respecto de la CPU.

4.4 Versión CUDA C++ clásica sin Tile

Además de la versión CUDA Tile C++, cada grupo deberá implementar una versión CUDA C++ clásica de la pipeline o de las tres etapas principales. Esta versión debe usar kernels tradicionales con grilla, bloques, hilos e indexación explícita, sin utilizar el modelo CUDA Tile.

El objetivo de esta versión es comparar dos formas de programar en GPU: control explícito de hilos y bloques versus programación por tiles. Esta comparación no implica que CUDA Tile deba ser siempre más rápido. El resultado puede mejorar, mantenerse o empeorar

según el tamaño de imagen, la forma del tile, el patrón de acceso a memoria, el compilador, la arquitectura de la GPU y las transferencias realizadas.

El grupo deberá explicar las diferencias observadas en correctitud, complejidad de implementación, indexación, memoria, medición y rendimiento entre CUDA C++ clásico y CUDA Tile C++.

4.5 Etapa complementaria en cuTile Python

Además de las implementaciones en CUDA C++ clásico y CUDA Tile C++, **cada grupo deberá implementar al menos una etapa en cuTile Python**. Puede ser Gaussian blur, Sobel, resize bilineal, conversión RGB a luminancia u otro kernel auxiliar justificado.

Esta etapa debe ser funcional, ejecutable y comparada contra la versión CPU, la versión CUDA C++ clásica o la versión CUDA Tile C++. El grupo deberá explicar cómo se lanzan los kernels desde Python, cómo se manejan los arreglos en GPU, qué datos se transfieren o comparten y qué diferencias observa respecto de las implementaciones en C++.

5. Versiones obligatorias a comparar

La actividad deberá incluir 4 versiones o componentes comparables:

1. CPU secuencial: referencia de correctitud y línea base de rendimiento.
2. CUDA C++ clásico sin Tile: implementación funcional de la pipeline completa usando kernels tradicionales con hilos, bloques, grilla e indexación explícita.
3. CUDA Tile C++ base: implementación funcional de la pipeline completa usando el modelo de programación por tiles.
4. cuTile Python: implementación funcional de una etapa o kernel auxiliar.

La comparación debe incluir resultados visuales, métricas de error, mediciones de tiempo, speed-up y explicación técnica de las diferencias observadas. El grupo debe explicar si CUDA Tile mejora, mantiene o empeora el rendimiento frente a CUDA C++ clásico; no se debe asumir que el modelo Tile siempre produce una mejora de rendimiento.

6. Requisitos técnicos transversales

Estos requisitos aplican a toda la pipeline y deben estar documentados en el informe, README y código entregado:

- Representación de la imagen en memoria: interleaved RGB o planar RGB.
- Indexación utilizada para acceder a píxeles y canales.
- Configuración de grilla, bloques e hilos en la versión CUDA C++ clásica.
- Tamaño y forma de los tiles en la versión CUDA Tile C++.
- Comparación técnica entre CUDA C++ clásico y CUDA Tile C++.
- Tratamiento de bordes en CPU y GPU.
- Transferencias host-device y device-host.
- Medición separada de tiempo total, tiempo de kernels y tiempo de transferencia usando herramientas de medición de CUDA.
- Profiling con Nsight Compute, Nsight Systems o herramienta equivalente del CUDA Toolkit.
- Registro reproducible de comandos, parámetros y herramientas de medición utilizadas.

No se aceptará como solución principal llamar directamente a funciones de **OpenCV, NPP, CuPy, PyTorch u otra biblioteca que ya implemente el filtro completo en GPU**. Se permite utilizar bibliotecas para lectura/escritura de imágenes, administración de arreglos o visualización, siempre que el procesamiento evaluado sea implementado por el grupo. Tampoco se aceptará una comparación de rendimiento basada solo en un cronómetro externo; las mediciones relevantes deben respaldarse con CUDA Events, Nsight Compute, Nsight Systems u otra herramienta equivalente del CUDA Toolkit.

7. Diseño experimental mínimo

Cada grupo deberá ejecutar experimentos suficientes para comparar correctitud, rendimiento y estabilidad. Como mínimo, se solicita:

- cuatro tamaños de imagen: pequeña, mediana y grande, no divisible.
- Una imagen con dimensiones no divisibles por el tamaño del tile.
- Dos configuraciones para el filtro gaussiano, por ejemplo 5x5 y 9x9, o dos valores de sigma.
- Dos factores de redimensionamiento, por ejemplo 0.5x y 1.75x.
- Comparación entre CPU secuencial, CUDA C++ clásico y CUDA Tile C++ base; además, comparación de la etapa cuTile Python contra una referencia equivalente.
- Una etapa implementada en cuTile Python y comparada contra CPU, CUDA C++ clásico o CUDA Tile C++.
- Al menos 10 repeticiones por configuración experimental relevante.
- Comparación explícita entre CUDA C++ clásico y CUDA Tile C++, indicando que una diferencia de modelo no garantiza por sí sola una mejora de rendimiento.
- Reporte de hardware, driver, CUDA Toolkit, nvcc, Python y paquetes utilizados.

Instancia	Dimensiones
Pequeña	512 x 512 o similar
Mediana	2048 x 2048 o similar
Grande	4096 x 4096 o similar
No divisible	1537 x 1021 o similar

8. Métricas obligatorias

El informe técnico, el CSV de resultados y, cuando corresponda, el README deberán incluir las siguientes métricas:

- Tiempo promedio de ejecución total.
- Desviación estándar del tiempo total.
- Tiempo de cada etapa de la pipeline: Gaussian blur, Sobel y resize bilineal.
- Tiempo de la versión CUDA C++ clásica sin Tile.
- Tiempo de kernels CUDA medido con CUDA Events, Nsight Compute, Nsight Systems o herramienta equivalente del CUDA Toolkit. (**Mostrar cuanto se demora por kernel**)
- Tiempo de transferencia host-device y device-host, cuando corresponda.
- Throughput en megapíxeles por segundo.
- Speed-up respecto de la versión CPU secuencial para CUDA C++ clásico y CUDA Tile C++ base.
- Comparación entre CUDA C++ clásico y CUDA Tile C++ base.
- Efecto del tamaño del tile o configuración equivalente.
- Efecto de la configuración de bloques, grilla o tile aplicada.
- Resultado de profiling con Nsight Compute, Nsight Systems o herramienta equivalente de CUDA.
- Evidencia de medición y profiling con herramientas CUDA, incluyendo al menos una observación sobre kernels, transferencias o cuellos de botella.

El speed-up debe calcularse comparando el tiempo de la versión CPU secuencial contra el tiempo de la versión GPU correspondiente. El grupo debe indicar explícitamente si el tiempo GPU incluye solo kernels o también transferencias entre host y device. El rendimiento debe

evaluarse con herramientas de medición de CUDA, por ejemplo CUDA Events para tiempos internos, Nsight Compute para análisis de kernels y Nsight Systems para flujo de ejecución y transferencias.

9. Entregables

1. Código fuente en CUDA C++ clásico sin Tile, CUDA Tile C++, C++ o Python para la referencia secuencial y cuTile Python para la etapa complementaria.
2. README con instrucciones claras de instalación, compilación, ejecución y reproducción de experimentos.
3. Imágenes de entrada utilizadas y, si corresponde, scripts para generarlas o descargarlas.
4. Imágenes de salida generadas por cada etapa de la pipeline.
5. Archivo CSV con resultados experimentales, incluyendo CPU, CUDA C++ clásico, CUDA Tile C++ base y cuTile Python cuando corresponda.
6. Archivo o carpeta con perfiles de Nsight Compute, Nsight Systems, registros de CUDA Events o capturas suficientes si el entorno no permite guardar perfiles.
7. Registro de comandos utilizados para ejecutar los experimentos y generar los resultados.
8. Informe en PDF.
9. Carpeta comprimida o repositorio organizado con todo el código, README, scripts de ejecución, resultados, perfiles y evidencias solicitadas.

La entrega del código es obligatoria. El grupo deberá asegurar que el programa compile o ejecute siguiendo el README, genere las imágenes de salida, produzca el CSV de resultados y permita reproducir las mediciones. Si el código no funciona, no se aceptará una explicación solo textual como reemplazo de la implementación.

10. Pauta de evaluación del informe técnico

La siguiente pauta corresponde al 70 % de la actividad. Cada criterio se evalúa con puntaje entero de 0 a 3 y luego se pondera según el porcentaje indicado dentro de esta pauta.

Criterio	Ponderación	0 puntos	1 punto	2 puntos	3 puntos
Introducción, contexto y formulación de la pipeline	15 %	No presenta el problema, no define la pipeline o la introducción es inexistente, confusa o desconectada de la actividad.	Presenta una descripción general del procesamiento de imágenes, pero no conecta claramente la actividad con paralelismo en GPU, CUDA C++ clásico, CUDA Tile C++ y cuTile Python.	Presenta la pipeline y sus etapas principales, aunque con omisiones en entradas, salidas, propósito de cada etapa, paralelismo de datos o dificultad técnica.	Contextualiza claramente el problema, la pipeline RGB, las etapas Gaussian blur, Sobel y resize bilineal, las entradas y salidas, el paralelismo por píxeles/tiles, la dificultad técnica y el sentido de comparar CPU, CUDA C++ clásico, CUDA Tile C++ y cuTile Python.
Fundamento matemático	15 %	No explica los operadores matemáticos, los explica incorrectamente o presenta fórmulas que no corresponden a la implementación realizada.	Describe superficialmente Gaussian blur, Sobel o resize bilineal, sin fórmulas completas, sin explicar variables, sin tratamiento de bordes ni criterios de validación.	Explica las etapas principales con fórmulas y conceptos generales, pero faltan detalles sobre normalización, luminancia, interpolación, precisión numérica, bordes o comparación con CPU.	Explica correctamente Gaussian blur, Sobel y resize bilineal; define kernel, sigma o tamaño de máscara, luminancia, gradientes Gx/Gy, magnitud, coordenadas fraccionales, interpolación, bordes, precisión esperada, tolerancias y criterios de aceptación entre CPU y GPU.
Metodología y diseño experimental	20 %	No presenta metodología experimental clara o solo muestra resultados sin explicar cómo fueron obtenidos.	Presenta experimentos insuficientes, sin control de imágenes, tamaños, variantes, repeticiones, parámetros, hardware, software o procedimiento de ejecución.	Presenta un diseño experimental aceptable, pero con omisiones en repeticiones, tamaños de imagen, parámetros del filtro, factores de resize, configuración de bloques/tiles, entorno CUDA o procedimiento de medición.	Define claramente imágenes usadas, tamaños, dimensiones no divisibles, parámetros de Gaussian, factores de resize, versiones comparadas, repeticiones, hardware, GPU, driver, CUDA Toolkit, nvcc, Python, paquetes, comandos, herramientas de medición, procedimiento experimental y trazabilidad completa de cada resultado.
Descripción técnica de CUDA C++ clásico, CUDA Tile C++ y cuTile Python	20 %	No describe la implementación CUDA ni Python, o la descripción es tan general que no permite entender qué fue programado.	Describe etapas aisladas, pero no explica kernels clásicos, tiles, memoria, transferencias, bordes, indexación, lanzamiento de kernels ni diferencias entre versiones.	Describe las implementaciones principales, pero con bajo detalle sobre bloques, grilla, hilos, tiles, shapes, views, datos, bordes, flujo CPU/GPU, memoria o comparación entre C++ y Python.	Explica técnicamente la representación de imagen, layout interleaved o planar, indexación de píxeles/canales, kernels CUDA clásicos, configuración de bloques/grilla, forma y tamaño de tiles, manejo de bordes, dimensiones no divisibles, transferencias host-device/device-host, etapa cuTile Python, comandos de ejecución y decisiones de diseño.
Resultados, tablas y gráficos	15 %	No presenta resultados cuantitativos, no muestra salidas visuales o los resultados no corresponden a las versiones solicitadas.	Presenta resultados incompletos, poco claros, sin tablas suficientes, sin gráficos relevantes o sin separar CPU, CUDA clásico, CUDA Tile y Python.	Presenta tablas, gráficos y salidas visuales suficientes, pero con interpretación limitada o ausencia de algunas métricas como desviación estándar, speed-up, throughput, error o tiempos por etapa.	Presenta resultados completos y reproducibles: imágenes de entrada/salida, comparación visual, tiempos promedio, desviación estándar, tiempos por etapa, tiempos de kernel, transferencias, speed-up, throughput, error o diferencia respecto a CPU, y comparación clara entre CPU, CUDA C++ clásico, CUDA Tile C++ y cuTile Python.
Análisis técnico, profiling y medición de rendimiento	10 %	No analiza rendimiento, no presenta profiling o solo entrega tiempos sin interpretación técnica.	Entrega una interpretación superficial, sin relacionar resultados con memoria, transferencias, configuración de bloques/tiles, overhead, profiling o herramientas CUDA.	Analiza parcialmente rendimiento, memoria, transferencias, kernels o profiling, pero con vacíos técnicos o sin conectar claramente las mediciones con las decisiones de implementación.	Discute críticamente el rendimiento, overhead, transferencias, tiempos de kernel, memoria, tamaño de imagen, configuración de bloques, forma de tile, diferencias entre CUDA clásico y CUDA Tile, etapa Python, profiling con Nsight Compute/Systems o herramienta CUDA equivalente, cuellos de botella, limitaciones y aprendizajes técnicos.
Redacción, orden y trazabilidad	5 %	El informe está desordenado, incompleto, difícil de seguir o no permite relacionar metodología, resultados y conclusiones.	Presenta problemas importantes de redacción, formato, numeración, tablas, gráficos, nombres de archivos o trazabilidad de resultados.	El informe es comprensible y tiene formato aceptable, aunque presenta inconsistencias menores en redacción, rotulado, referencias internas, parámetros o correspondencia con los archivos entregados.	Informe claro, ordenado y bien redactado; usa secciones coherentes, tablas y gráficos rotulados, unidades correctas, parámetros trazables, nombres de archivos consistentes, comandos o anexos verificables, y coherencia entre metodología, resultados, análisis, conclusiones y código entregado.

11. Pauta de evaluación del código fuente y reproducibilidad

La siguiente pauta corresponde al 30 % de la actividad.

Criterio	Ponderación	0 puntos	1 punto	2 puntos	3 puntos
Entrega real del código fuente, estructura del proyecto, compilación y ejecución base	5 %	No entrega código fuente, entrega solo resultados, solo imágenes, solo ejecutables, archivos incompletos o un proyecto que no puede compilarse ni ejecutarse por errores atribuibles al grupo.	Entrega código fuente, pero desordenado, con rutas absolutas, dependencias no declaradas, archivos faltantes o necesidad de modificaciones importantes para intentar compilar o ejecutar.	El proyecto compila y ejecuta con ajustes menores; incluye una estructura básica y un README parcial, pero faltan detalles de dependencias, entorno, comandos o parámetros.	Proyecto completo, ordenado y reproducible desde una carpeta limpia; incluye README claro, estructura de carpetas, dependencias, comandos de compilación y ejecución para CPU, CUDA C++ clásico, CUDA Tile C++ y cuTile Python.
Pipeline funcional de procesamiento de imágenes	10 %	El programa no procesa ninguna imagen de entrada de forma funcional, genera salidas inválidas o solo entrega archivos previamente generados sin ejecución verificable.	Procesa parcialmente la imagen, pero falta una o más etapas principales, hay errores relevantes en dimensiones, formato RGB, guardado de salidas o flujo entre etapas.	Implementa la mayor parte de la pipeline, aunque con omisiones menores en parámetros, nombres de salida, control de etapas o manejo de algunos casos borde.	Implementa una pipeline completa y funcional sobre imágenes RGB, incluyendo Gaussian blur, Sobel y resize bilineal; permite ejecutar cada versión, configurar parámetros relevantes y generar salidas verificables por etapa y versión.
Referencia CPU secuencial y validación de precisión	10 %	No existe versión CPU secuencial, la versión CPU no corresponde al mismo algoritmo evaluado o no se usa como referencia de correctitud.	La referencia CPU existe solo para una parte del programa o no permite comparar de forma confiable contra las versiones GPU.	La referencia CPU cubre la mayoría de las etapas y se compara parcialmente con GPU, pero faltan métricas, tolerancias, validación por etapa o tratamiento de bordes consistente.	Implementa referencia CPU secuencial completa para Gaussian blur, Sobel y resize bilineal; compara CPU vs CUDA clásico, CPU vs CUDA Tile y la etapa Python cuando corresponda; presenta validación visual coherente y criterios claros para aceptar diferencias entre versiones.
Implementación CUDA C++ clásica sin Tile	15 %	No existe implementación CUDA C++ clásica funcional o la versión entregada no ejecuta trabajo real en GPU.	Presenta kernels incompletos, superficiales o con errores relevantes de indexación, configuración de grilla/bloques, accesos fuera de rango, memoria o transferencia de datos.	Implementa la mayoría de las etapas en CUDA C++ clásico, pero con limitaciones en control de bordes, dimensiones no divisibles, separación de etapas, configuración de bloques o explicación técnica del flujo CPU/GPU.	Implementa el programa completo en CUDA C++ clásico sin Tile, usando kernels tradicionales, bloques, grilla, hilos, indexación explícita, control de límites, tratamiento de bordes, transferencias host-device/device-host y validación por etapa.
Implementación CUDA Tile C++	20 %	No existe implementación CUDA Tile C++ funcional o se presenta una versión que no usa realmente el modelo de programación por tiles.	La versión Tile es incompleta, decorativa o solo replica código clásico sin evidenciar organización por tiles, manejo de bordes ni comparación real.	Implementa CUDA Tile C++ en las etapas principales, pero con limitaciones en forma/tamaño de tile, dimensiones no divisibles, bordes, validación o comparación con CUDA clásico.	Implementa el programa completo usando CUDA Tile C++; define tamaño y forma de tiles, maneja bordes y dimensiones no divisibles, justifica decisiones de diseño y compara técnicamente con CUDA C++ clásico sin asumir mejora automática de rendimiento.
Etapas complementarias en cuTile Python	10 %	No entrega etapas funcionales en cuTile Python o entrega un script Python que no ejecuta procesamiento relacionado con la actividad.	Entrega una etapa Python aislada, difícil de ejecutar, sin integración con los datos de la actividad o sin comparación con una referencia equivalente.	Implementa una etapa funcional en cuTile Python, pero con validación limitada, parámetros rígidos o explicación incompleta del manejo de arreglos y ejecución en GPU.	Implementa y ejecuta una etapa en cuTile Python, por ejemplo Gaussian, Sobel, resize o luminancia; compara contra CPU, CUDA C++ clásico o CUDA Tile; documenta entrada/salida, arreglos GPU, lanzamiento, transferencias y limitaciones observadas.
Implementación propia de los kernels y uso permitido de bibliotecas	5 %	Resuelve directamente los filtros evaluados usando OpenCV, NPP, CuPy, PyTorch u otra biblioteca que ya implemente el procesamiento en GPU; o el código principal no permite identificar qué fue implementado por el grupo.	Usa bibliotecas externas de forma excesiva o poco clara, mezclando procesamiento propio con funciones ya resueltas, sin separación ni justificación suficiente.	Usa bibliotecas principalmente para lectura, escritura, visualización o manejo auxiliar, aunque existen partes del procesamiento que requieren mejor justificación o separación.	El procesamiento evaluado está implementado por el grupo; las bibliotecas se usan solo para lectura/escritura, visualización, carga de datos o utilidades auxiliares; cualquier código externo adaptado está claramente citado y modificado con criterio técnico.
Manejo de memoria, datos, bordes y errores CUDA	10 %	El código presenta errores graves de memoria, accesos fuera de rango, fugas evidentes, ausencia total de control de errores CUDA o resultados corruptos.	Maneja memoria y datos de forma básica, pero con errores o ambigüedades en layout RGB, índices, bordes, sincronización, liberación de memoria o chequeo de errores.	El manejo de memoria es funcional, pero faltan detalles en sincronización, revisión de errores, consistencia de tipos, layout de imagen o casos con dimensiones no divisibles.	Define claramente layout interleaved o planar, índices de píxeles y canales, memoria host/device, copias, sincronización, liberación de recursos, chequeo de errores CUDA, bordes, dimensiones no divisibles y consistencia numérica entre CPU y GPU.
Medición de rendimiento y profiling integrado	10 %	No mide tiempos de forma confiable, usa solo cronómetro externo o presenta mediciones que no pueden reproducirse desde el código.	Mide tiempos de manera parcial, sin separar etapas, kernels, transferencias, repeticiones o versiones comparadas.	Usa CUDA Events, Nsight Compute, Nsight Systems o herramienta equivalente, pero con separación incompleta de métricas, pocas repeticiones o exportación limitada de resultados.	Integra medición reproducible con CUDA Events y/o herramientas CUDA; separa tiempo total, kernels, transferencias y etapas; ejecuta repeticiones, calcula promedio, desviación estándar, speed-up, throughput y permite comparar CPU, CUDA clásico, CUDA Tile y Python.
Generación de resultados, CSV y trazabilidad experimental	5 %	Los resultados, imágenes o CSV entregados no pueden regenerarse desde los comandos indicados; los archivos de salida no son evidencia válida porque no existe trazabilidad con el código.	Genera algunos archivos de salida, pero sin nombres claros, sin relación verificable con parámetros, versiones, imágenes o configuración experimental.	Genera imágenes y CSV reproducibles parcialmente, aunque faltan campos importantes como versión, etapa, tamaño, bloque, tile, repetición, tiempo o error.	Genera automáticamente imágenes de salida, CSV y registros de ejecución; cada resultado indica versión, etapa, imagen, dimensiones, kernel/sigma, factor de escala, bloque, tile, repetición, tiempo, error y herramienta de medición utilizada.